

Outline

We will now look at our first abstract data structure

- Relation: explicit linear ordering
- Operations
- Implementations of an abstract list with:
 - Linked lists
 - Arrays
- Memory requirements
- Strings as a special case
- The STL vector class

Definition

An Abstract List (or List ADT) is linearly ordered data where the programmer explicitly defines the ordering

We will look at the most common operations that are usually

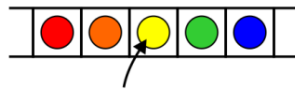
- The most obvious implementation is to use either an array or linked list
- These are, however, not always the most optimal

Lists

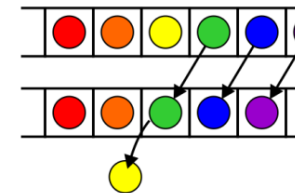
Operations

Operations at the k^{th} entry of the list include:

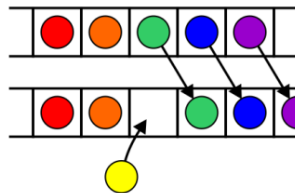
Access to the object



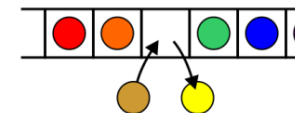
Removal of the object



Insertion of a new object



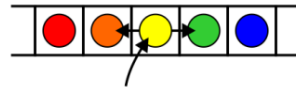
Replacement of the object



Lists

Operations

Given access to the k^{th} object, gain access to either the previous or next object



Given two abstract lists, we may want to

- Concatenate the two lists
- Determine if one is a sub-list of the other

Data Structures

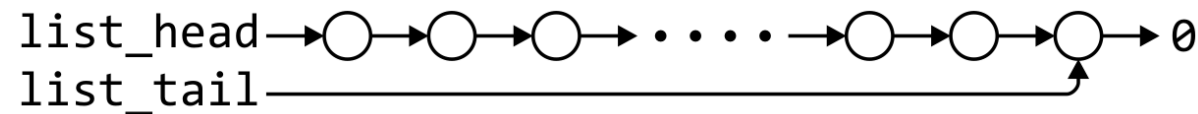
The most obvious data structures for implementing an abstract list are arrays and linked lists

- We will examine each of these with respect to the operations
- We will consider if the k^{th} object is:
 - At front of the list (first)
 - At an arbitrary position, and
 - At the back of the list (n^{th})
- We will conclude by looking at other operations

Lists

Singly Linked Lists

We begin with a singly linked list with only a head pointer

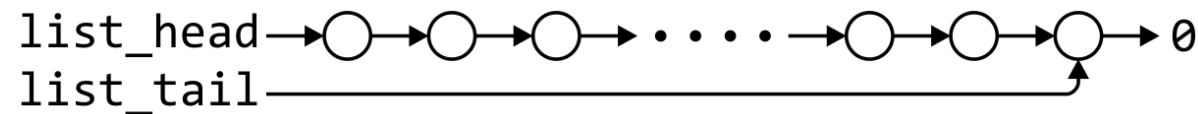


	Front/ 1^{st}	Back/ n^{th}
Find	$\Theta(1)$	$\Theta(1)$
Insert Before	$\Theta(1)$	$\Theta(n)$
Insert After	$\Theta(1)$	$\Theta(1)$
Replace	$\Theta(1)$	$\Theta(1)$
Erase	$\Theta(1)$	$\Theta(n)$
Next	$\Theta(1)$	-
Previous	-	$\Theta(n)$

Lists

Singly Linked Lists

To get a pointer to the k^{th} node, we must find it: $O(n)$



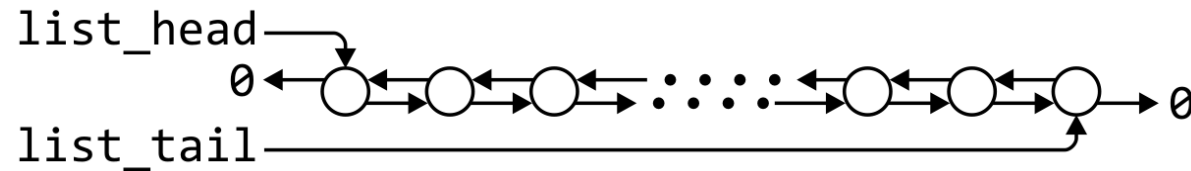
– Now, suppose we have found that pointer

	k^{th} Node
Insert Before	$O(n)$
Insert After	$\Theta(1)$
Replace	$\Theta(1)$
Erase	$O(n)$
Next	$\Theta(1)$
Previous	$O(n)$

Lists

Doubly Linked Lists

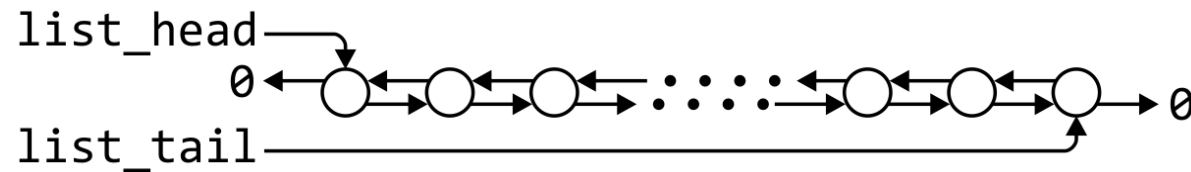
We begin with a singly linked list with only a head pointer



	Front/ 1^{st}	Back/ n^{th}
Find	$\Theta(1)$	$\Theta(1)$
Insert Before	$\Theta(1)$	$\Theta(1)$
Insert After	$\Theta(1)$	$\Theta(1)$
Replace	$\Theta(1)$	$\Theta(1)$
Erase	$\Theta(1)$	$\Theta(1)$
Next	$\Theta(1)$	-
Previous	-	$\Theta(1)$

Doubly Linked Lists

To get a pointer to the k^{th} node, it is still expensive: $O(n)$



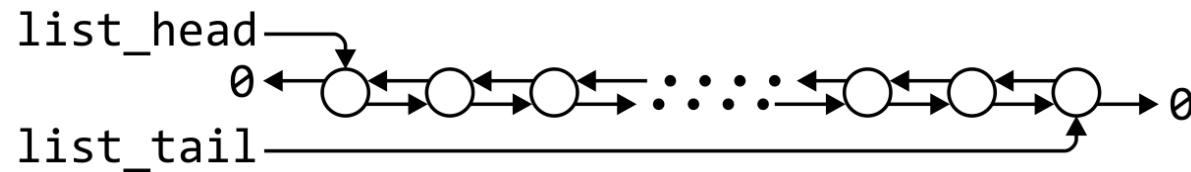
- Now, suppose we have found that pointer

	k^{th} Node
Insert Before	$O(1)$
Insert After	$\Theta(1)$
Replace	$\Theta(1)$
Erase	$O(1)$
Next	$\Theta(1)$
Previous	$O(1)$

Lists

Linked Lists

Unfortunately, finding the k^{th} object will always be expensive



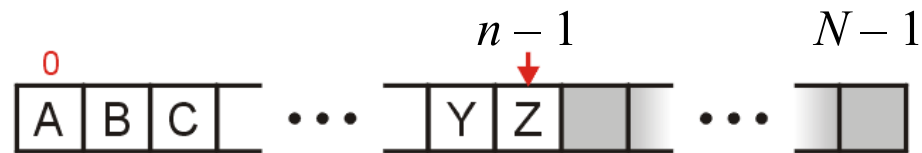
Other operations:

- Concatenating two linked lists with tail pointers is $\Theta(1)$
- Deleting a linked list is $\Theta(n)$

Arrays

The following model uses an array to implement an abstract list:

- Allocate N memory locations (the *capacity* of the array)
- As the array is filled with n objects, keep track of the last object



- The objects can be shifted to make room for new ones

Lists

Arrays

Assuming the array is not full, only insertions and erases at the front or at an arbitrary location are slow

	Front/1 st	k^{th}	Back/ n^{th}
Find	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Insert Before	$\Theta(n)$	$O(n)$	$\Theta(1)$
Insert After	$\Theta(n)$	$O(n)$	$\Theta(1)$
Replace	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Erase	$\Theta(n)$	$O(n)$	$\Theta(1)$
Next	$\Theta(1)$	$\Theta(1)$	-
Previous	-	$\Theta(1)$	$\Theta(1)$

Arrays

If the array is full, insertions are not possible...

In the description of an abstract list, there is no restriction on the number of objects which can be stored

- This is an *implementation* error

There are three possible solutions:

- Throw an exception
- Ignore the insertion
- Increase the size of the array

Lists

Arrays

Increasing the size of the array is in many cases the best solution

- The question is, by how much do we increase the array?
- Two possibly solutions:
 - Double the size of the array
 - Increase the size of the array by some fixed value
- We shall see which is better when using arrays to implement *Abstract Stacks*

Lists

Arrays

Some comments on other operations includes:

- Finding an object is expensive: $O(n)$
- If the list is sorted, we can find an object with a binary search: $O(\ln(n))$
- Concatenating two arrays is $\Theta(n + m)$
- Deleting an array for built-in data types is $\Theta(1)$

Arrays

Another issue are the $\Theta(n)$ operations at the front of the array

- Can we reduce these run times?

	Front/ 1^{st}	k^{th}	Back/ n^{th}
Find	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Insert Before	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Insert After	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Replace	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Erase	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Next	$\Theta(1)$	$\Theta(1)$	-
Previous	-	$\Theta(1)$	$\Theta(1)$

Two-ended Arrays

Consider an array which we begin filling in the middle

Given array of capacity N , start insertions at $N/2$

- We can make insertions on whichever side requires fewer copies



We will call this a *two-ended array*

- To differentiate, we will call the standard array a *one-ended array*

Two-ended Arrays

What happens if we fill up a two-ended array at one end and make an insertion?



Two-ended Arrays

What happens if we fill up a two-ended array at one end and make an insertion?

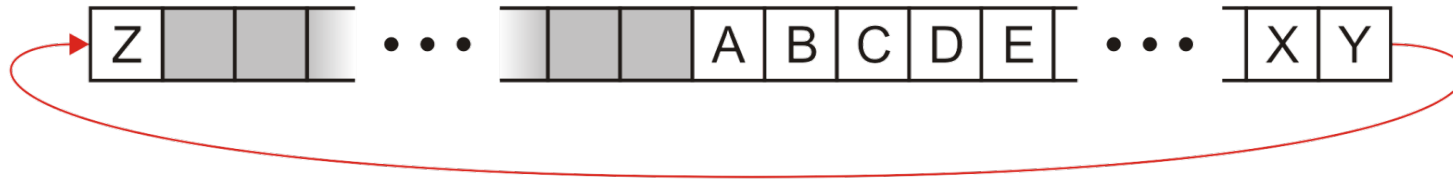
- The expensive operation would be to copy everything over to make a place for the new entry



Two-ended Arrays

What happens if we fill up a two-ended array at one end and make an insertion?

- As we have room at the other end, why not use it?
- This, however, requires more book-keeping



Memory Usage

All of these data structures require $\Theta(n)$ memory

- Using a two-ended array requires one more member variable, $\Theta(1)$, in order to significantly speed up certain operations
- Using a doubly linked list, however, required $\Theta(n)$ additional memory to speed up other operations

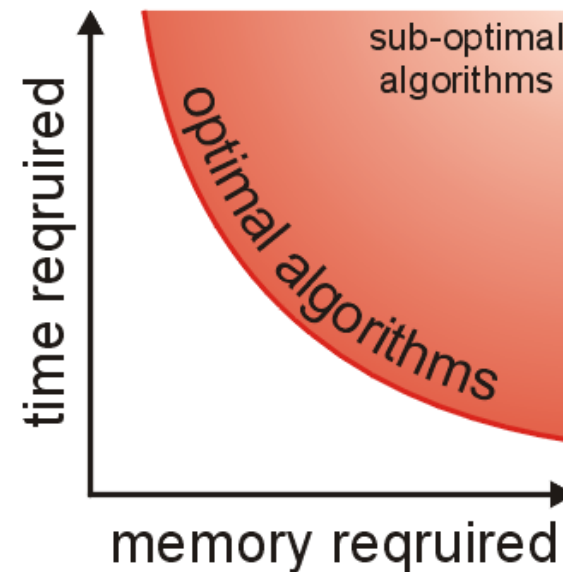
Memory Usage

As well as determining run times, we are also interested in memory usage

In general, there is an interesting relationship between memory and time efficiency

For a data structure/algorithm:

- Improving the run time usually requires more memory
- Reducing the required memory usually requires more run time



Memory Usage

Warning: programmers often mistake this to suggest that given any solution to a problem, any solution which may be faster must require more memory

This guideline not true in general: there may be different data structures and/or algorithms which are both faster and require less memory

- This requires thought and research

The **sizeof** Operator

In order to determine memory usage, we must know the memory usage of the various primitive data types and classes

- The **sizeof** operator in C++ returns the number of bytes occupied by a data type
- This value is determined at compile time
 - It is **not** a function

The `sizeof` Operator

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "bool      " << sizeof( bool )    << endl;
    cout << "char      " << sizeof( char )    << endl;
    cout << "short     " << sizeof( short )   << endl;
    cout << "int       " << sizeof( int )     << endl;
    cout << "char *    " << sizeof( char * )  << endl;
    cout << "int *     " << sizeof( int * )   << endl;
    cout << "double    " << sizeof( double )  << endl;
    cout << "int[10]   " << sizeof( int[10] ) << endl;

    return 0;
}
```

```
{eceunix:1} ./a.out # output
```

```
bool      1
char      1
short     2
int       4
char *    4
int *     4
double    8
int[10]   40
```

```
{eceunix:2}
```

Abstract Strings

A specialization of an Abstract List is an Abstract String:

- The entries are restricted to *characters* from a finite *alphabet*
- This includes regular strings “Hello world!”

The restriction using an alphabet emphasizes specific operations that would seldom be used otherwise

- Substrings, matching substrings, string concatenations

It also allows more efficient implementations

- String searching/matching algorithms
- Regular expressions

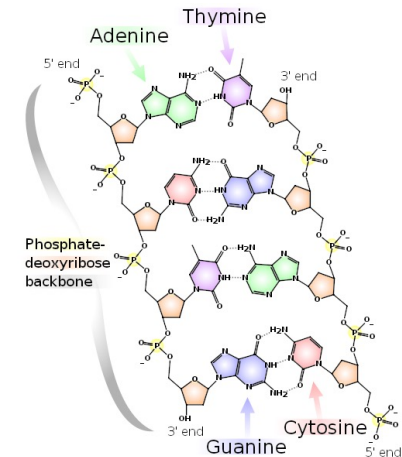
Abstract Strings

Strings also include DNA

- The alphabet has 4 *characters*: A, C, G, and T
- These are the nucleobases:
adenine, cytosine, guanine, and thymine

Bioinformatics today uses many of the algorithms traditionally restricted to computer science:

- Dan Gusfield, *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge, 1997
<http://books.google.ca/books?id=STGlsyqtjYMC>
- References:
<http://en.wikipedia.org/wiki/DNA>
<http://en.wikipedia.org/wiki/Bioinformatics>



Standard Template Library

In this course, you must understand each data structure and their associated algorithms

- In industry, you will use other implementations of these structures

The C++ Standard Template Library (STL) has an implementation of the vector data structure

- Excellent reference:

<http://www.cplusplus.com/reference/stl/vector/>

Standard Template Library

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v( 10, 0 );

    cout << "Is the vector empty? " << v.empty() << endl;
    cout << "Size of vector: " << v.size() << endl;

    v[0] = 42;
    v[9] = 91;

    for ( int k = 0; k < 10; ++k ) {
        cout << "v[" << k << "] = " << v[k] << endl;
    }

    return 0;
}
```

```
[ece250@eceLinux1 ~]$ g++ vec.cpp
[ece250@eceLinux1 ~]$ ./a.out
Is the vector empty? 0
Size of vector: 10
v[0] = 42
v[1] = 0
v[2] = 0
v[3] = 0
v[4] = 0
v[5] = 0
v[6] = 0
v[7] = 0
v[8] = 0
v[9] = 91
[ece250@eceLinux1 ~]$
```

Summary

In this topic, we have introduced Abstract Lists

- Explicit linear orderings
- Implementable with arrays or linked lists
 - Each has their limitations
 - Introduced modifications to reduce run times down to $\Theta(1)$
- Discussed memory usage and the `sizeof` operator
- Looked at the String ADT
- Looked at the vector class in the STL

References

- [1] Donald E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd Ed., Addison Wesley, 1997, §2.2.1, p.238.
- [2] Weiss, *Data Structures and Algorithm Analysis in C++*, 3rd Ed., Addison Wesley, §3.3.1, p.75.